

Rapport detaille d'utilisation de KAI

Projet : modern_notification

Objectif : creer un composant Delphi/VCL affichant des notifications Windows 10/11 avec icones PNG incorporees dans le BPL.

Chemin projet : C:\Users\

<utilisateur>\Documents\Embarcadero\Studio\Projets\modern_notification

Projet RAD Studio : modern_notification.dproj

Package : modern_notification.dpk

Etat final releve : Win32 / Release , compilation complete OK.

1. Presentation du nouvel outil Embarcadero KAI

KAI est le nouvel assistant integre a RAD Studio pour accompagner le developpeur directement dans son environnement Delphi. Son interet principal est de travailler avec le projet ouvert dans l'IDE : il peut analyser le package actif, lire les fichiers ouverts, respecter les modifications non sauvegardees, ajouter des fichiers au projet et lancer les compilations.

Dans ce projet, KAI a ete utilise comme assistant de developpement pour transformer une simple procedure de notification en composant Delphi installable, avec ressources integrees, editeurs design-time, icones de palette et rapport de suivi.

Points forts observes

- **Integration IDE-first** : KAI lit les buffers RAD Studio ouverts avant de lire les fichiers depuis le disque.
- **Respect des modifications non sauvegardees** : les edits sont appliques dans l'editeur quand le fichier est ouvert.
- **Gestion du projet actif** : KAI connait le package, la plateforme, la configuration et les fichiers declares dans le projet.
- **Edition ciblee** : les changements peuvent etre appliques avec precision via `applyEdit` , sans reecrire inutilement tout le projet.
- **Ajout de fichiers au package** : les unites et ressources peuvent etre ajoutees au projet avec `addFileToProject` .
- **Validation continue** : les compilations sont lancees depuis RAD Studio avec `compileProjects` .
- **Assistance technique contextuelle** : KAI a adapte le code aux contraintes Delphi, VCL, design-time et BPL.
- **Tracabilite** : les decisions et etapes peuvent etre documentees dans un rapport reproductible.

Operations KAI utilisees dans ce projet

Outil KAI	Utilisation dans le projet
<code>getProjectInfo</code>	Identifier le projet actif, la plateforme, la configuration, le type de framework et la liste des fichiers du package.
<code>listOpenFiles</code>	Savoir quels fichiers etaient ouverts dans RAD Studio avant lecture ou modification.
<code>getEditorContent</code> / <code>getEditorLines</code>	Lire le contenu reel des buffers IDE, y compris les changements non sauvegardes.
<code>getModificationStatus</code>	Verifier si un fichier etait ouvert et modifie avant d'appliquer une edition.
<code>setEditorContent</code>	Remplacer l'unite runtime initiale lorsque le composant a ete cree.
<code>applyEdit</code>	Appliquer des modifications ciblees dans les fichiers ouverts par l'IDE, notamment <code>notify_Unit.pas</code> et <code>modern_notification.dpk</code> .
<code>addFileToProject</code>	Ajouter les nouveaux fichiers au projet RAD Studio : unite design-time et fichiers ressources.
<code>compileProjects</code>	Compiler regulierement le package pour verifier chaque etape.

Outils de modification des fichiers sources

Pour les fichiers sources ouverts dans RAD Studio, la modification a ete faite avec les outils KAI de l'IDE, principalement `applyEdit` pour les changements cibles et `setEditorContent` pour les remplacements complets. Cette methode permet de modifier le buffer de l'editeur sans perdre les changements non sauvegardes.

PowerShell a ete utilise en complement, mais pas comme outil principal pour modifier les sources ouvertes dans l'IDE. Il a servi aux operations systeme et aux controles autour du projet.

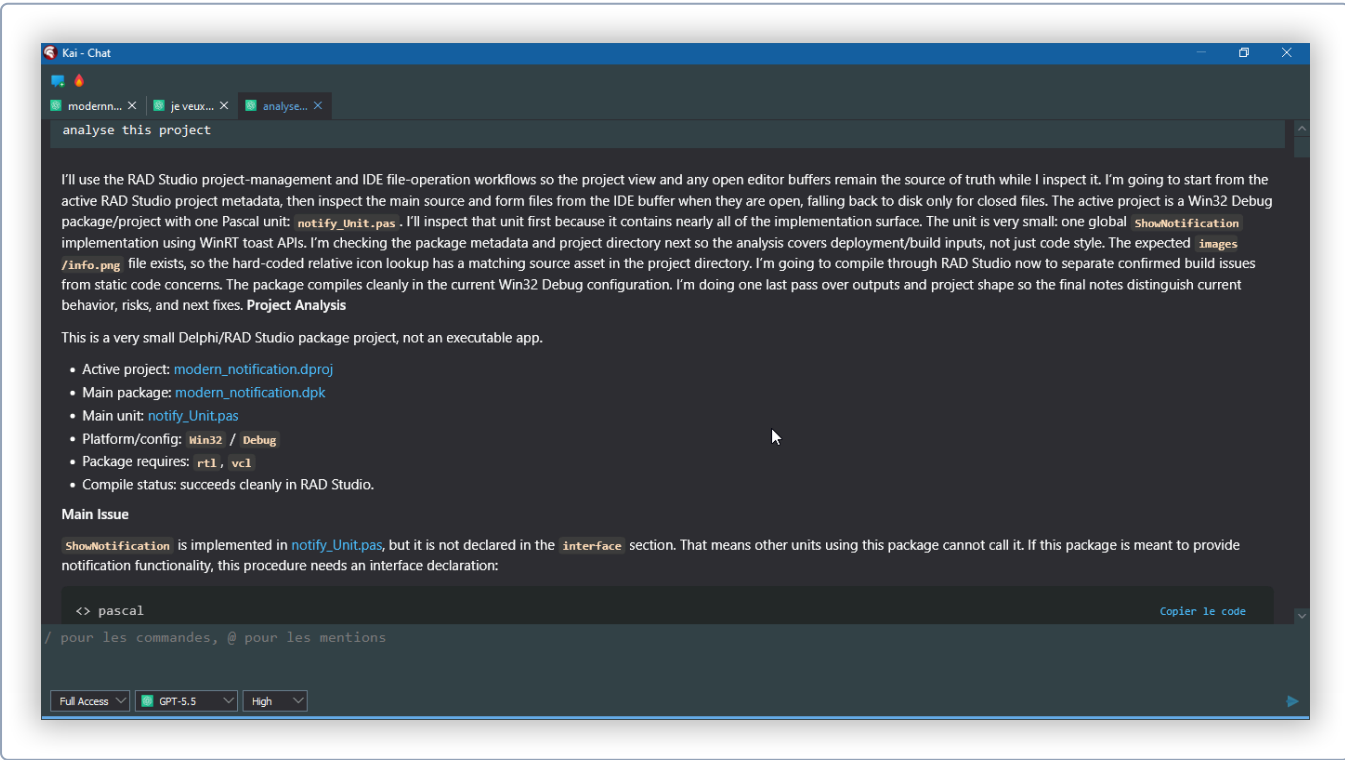
Outil	Role exact dans ce projet
KAI <code>applyEdit</code>	Modifier une portion precise d'un fichier ouvert dans RAD Studio, par exemple dans <code>notify_Unit.pas</code> ou <code>modern_notification.dpk</code> .
KAI <code>setEditorContent</code>	Remplacer completement le contenu d'un fichier ouvert lorsque c'etait plus simple et controle, notamment pour les petits fichiers de ressources.

Outil	Role exact dans ce projet
KAI addFileToProject	Declarer une nouvelle unite ou ressource dans le projet RAD Studio.
PowerShell	Lister les fichiers, verifier les dimensions des images, generer des bitmaps intermediaires, verifier les ressources du BPL, anonymiser une capture et convertir le HTML en PDF.

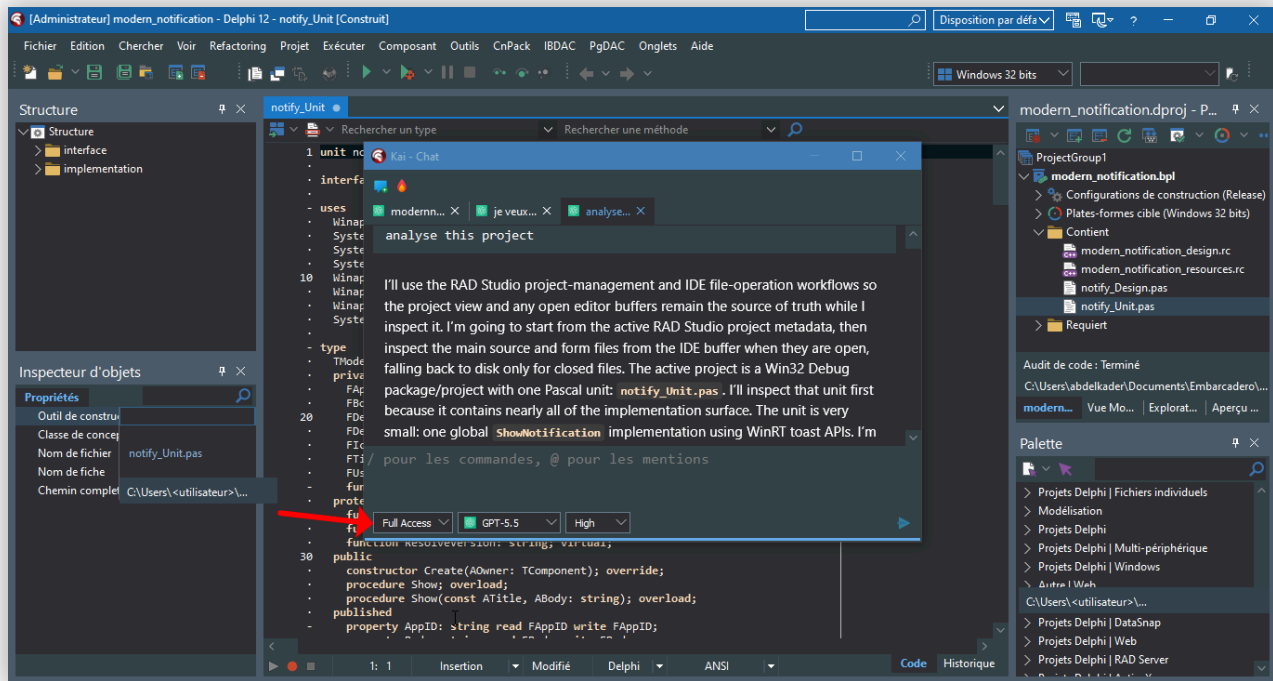
Principe applique : lorsque le fichier etait ouvert dans RAD Studio, KAI a ete utilise comme source de verite. Les acces disque ont ete reserves aux fichiers fermes, aux controles de ressources et a la generation de ce rapport.

2. Captures ecran

Les captures suivantes proviennent du dossier `shots`. Elles documentent visuellement l'utilisation de KAI pendant la creation et l'analyse du composant dans RAD Studio.



Capture 1 - Analyse initiale du projet dans la fenetre KAI.



Capture 2 - KAI utilise en parallele de RAD Studio avec le package modern_notification ouvert.

3. Discussion developpeur-KAI

Cette section reprend la discussion fonctionnelle qui a guide la construction du projet. Elle ne liste pas chaque appel technique en detail, mais elle conserve les demandes du developpeur, les decisions prises avec KAI et les validations effectuees.

Etape	Demande du developpeur	Reponse / action realisee avec KAI
1	Analyser le projet existant.	KAI a lu les metadonnees du projet actif, les fichiers ouverts et l'unite principale. Le projet a ete identifie comme un package Delphi contenant une procedure toast initiale.
2	Creer un composant Delphi affichant une notification Windows 10/11 avec grande icone.	Creation de <code>TModernNotification</code> , ajout des proprietes publiees, methode <code>Show</code> , procedure globale <code>ShowNotification</code> et enregistrement dans la palette.
3	Incorporer <code>info.png</code> dans le BPL.	Creation de <code>modern_notification_resources.rc</code> , liaison de la ressource au package et extraction temporaire de la ressource PNG au moment d'afficher le toast.

Etape	Demande du developpeur	Reponse / action realisee avec KAI
4	Permettre une icone developpeur 64x64, sinon utiliser l'icone incorporee.	La resolution d'icone a ete organisee pour privilegier <code>IconFileName</code> quand le fichier existe, puis retomber sur l'icone par default incorporee.
5	Ajouter <code>notification.png</code> comme petite icone du composant.	Conversion en bitmap compatible palette Delphi et declaration d'une ressource <code>TMODERNNOTIFICATION</code> pour l'icone design-time.
6	Ajouter la propriete <code>DefaultIcon</code> .	Ajout d'une propriete booleenne publiee, initialisee a <code>True</code> . Si elle vaut <code>False</code> , aucune icone incorporee n'est utilisee en fallback.
7	Utiliser une boite de dialogue pour le chemin d'icone, comme avec <code>TImage</code> .	Creation de <code>notify_Design.pas</code> et d'un editeur de propriete pour <code>IconFileName</code> , avec bouton <code>...</code> et <code>TOpenDialog</code> filtre sur les images.
8	Corriger l'icone par default et la petite icone du composant qui ne s'affichaient pas.	Correction du chargement de ressource avec <code>FindClassHInstance(TModernNotification)</code> et inclusion de la ressource design-time dans <code>notify_Design.pas</code> .
9	Expliquer l'utilite de <code>UseLargeIcon</code> .	Clarification : cette propriete force l'image comme grande icone toast avec <code>placement="appLogoOverride"</code> et <code>hint-crop="none"</code> .
10	Incorporer plus d'images PNG 64x64 du dossier <code>images</code> .	Ajout d'une table d'icomes incorporees, d'une propriete <code>DefaultIconName</code> et d'une liste deroulante design-time.
11	Ajouter les nouvelles images 64x64 du dossier <code>images</code> .	Synchronisation de <code>modern_notification_resources.rc</code> et de la table <code>EmbeddedIcons</code> . Verification du BPL : 27 ressources <code>MODERN_NOTIFICATION_ICON_*</code> .
12	Nettoyer le code des fonctions inutilisees.	Controle des references, conservation des fonctions utilisees par les proprietes ou le design-time, suppression de deux unites inutiles : <code>Winapi.WinRT</code> et <code>System.TypeInfo</code> .
13	Creer un rapport PDF detaille des etapes d'utilisation de KAI.	Generation d'un rapport HTML puis conversion en PDF avec Microsoft Edge headless. Compilation finale du package verifiee avant generation.

Extrait synthétique de la conversation

Developpeur : analyse this project
KAI : Inspection du projet actif, des fichiers ouverts et compilation de base.

Developpeur : le but c'est de creer un composant delphi qui affiche une notification.
KAI : Creation de TModernNotification, proprietes publiees et methode Show.

Developpeur : je veux que info.png soit incorporee dans le bpl.
KAI : Ajout d'une ressource RCDATA et extraction depuis le BPL.

Developpeur : le developpeur peut utiliser une icone 64x64, sinon l'icone incorporee.
KAI : Priorite a IconFileName, fallback sur l'icone incorporee.

Developpeur : ajouter DefaultIcon true/false.
KAI : Ajout de la propriete DefaultIcon et gestion de l'absence d'icone.

Developpeur : pourquoi ne pas utiliser la boite de dialogue comme TImage ?
KAI : Ajout d'un editeur de propriete design-time avec bouton "...".

Developpeur : l'icone par defaut et la petite icone ne s'affichent pas.
KAI : Correction du handle de module et de la ressource palette.

Developpeur : incorporer plus d'images PNG 64x64.
KAI : Ajout de DefaultIconName, liste deroulante et ressources multiples.

Developpeur : nettoyer le code.
KAI : Suppression des uses inutiles et recompilation.

Developpeur : creer un rapport PDF.
KAI : Generation du present rapport et conversion PDF.

4. Etapes de creation du composant

Les sections suivantes decrivent les etapes techniques realisees apres l'analyse initiale : transformation en composant, gestion des notifications Windows, ressources incorporees, design-time, validations et maintenance.

4.1 Analyse initiale du projet

La premiere etape a consiste a inspecter le package existant. KAI a identifie un projet package Delphi contenant principalement `notify_Unit.pas`, `modern_notification.dpk`, `modern_notification.dproj` et les ressources associees.

L'unite initiale contenait une procedure globale `ShowNotification` utilisant les API WinRT `TToastNotificationManager`, `IToastNotification` et les templates Windows toast. Cette procedure n'etait pas encore exposee comme composant installable.

KAI a ensuite compile le projet pour etablir une base de depart. La compilation initiale etait valide, ce qui a permis d'ajouter les fonctions progressivement avec une verification a chaque etape.

4.2 Transformation en composant Delphi

Le code runtime a été restructuré autour d'un composant non visuel :

```
type
  TModernNotification = class(TComponent)
  published
    property AppID: string;
    property Body: string;
    property DefaultIcon: Boolean;
    property DefaultIconName: string;
    property IconFileName: string;
    property Title: string;
    property UseLargeIcon: Boolean;
    property Version: string;
  public
    procedure Show; overload;
    procedure Show(const ATitle, ABody: string); overload;
  end;
```

KAI a remplacé le contenu de `notify_unit.pas`, puis a compilé le package. La procédure globale `ShowNotification` a été conservée pour permettre une utilisation simple sans déposer le composant sur une fiche.

4.3 Affichage des notifications Windows

Le composant construit un XML toast à partir des templates Windows :

- `ToastImageAndText02` lorsque l'icône existe.
- `ToastText02` lorsqu'aucune icône ne doit être affichée.

Les textes sont injectés dans les nœuds XML `text`. Si une image est utilisée, elle est convertie en URI locale `file:///...` puis affectée au nœud `image`.

La propriété `UseLargeIcon` ajoute :

```
placement="appLogoOverride"
hint-crop="none"
```

Cela permet d'obtenir une grande icône de notification dans le style Windows 10/11.

4.4 Icône par défaut incorporée dans le BPL

La demande suivante était d'incorporer `info.png` dans le BPL. Un fichier ressource `modern_notification_resources.rc` a été ajouté au projet, puis compilé en `modern_notification_resources.res`.

Le code runtime extrait la ressource PNG dans le dossier temporaire de Windows, car les notifications toast utilisent une URI de fichier pour les images locales. Le chargement de ressource utilise le handle du BPL contenant la classe, et non un `HInstance` ambigu :

```
LModuleHandle := FindClassHInstance(TModernNotification);  
LResourceStream := TResourceStream.Create(LModuleHandle, ResourceName, RT_RCDATA);
```

Cette correction a ete necessaire pour que la ressource soit retrouvee correctement apres installation du package.

4.5 Icone externe ou icone incorporee

Le composant prend en charge deux strategies :

- `IconFileName` : le developpeur choisit une image externe, par exemple une icone PNG 64x64.
- `DefaultIcon` et `DefaultIconName` : le composant utilise une icone incorporee dans le BPL.

La resolution finale suit cet ordre :

1. Si `IconFileName` pointe vers un fichier existant, ce fichier est utilise.
2. Sinon, si `DefaultIcon=False`, aucune icone n'est utilisee.
3. Sinon, la ressource indiquee par `DefaultIconName` est extraite et utilisee.
4. Si le nom est vide ou invalide, le composant revient a l'icone par default `button-info`.

4.6 Incorporation des PNG 64x64

Les images PNG 64x64 du dossier `images` ont ete declarees comme ressources `RCDATA`. Le composant garde une table interne associant :

- le nom visible dans `DefaultIconName`,
- le nom de ressource Windows,
- le nom du fichier temporaire extrait.

Exemples de noms exposes dans la liste :

```
button-cancel  
button-download  
button-error  
button-help  
button-info  
button-ok  
button-refresh  
button-trend-down  
button-update  
chart-pie-2d  
chart-pie-2d-exploded  
charts  
charts-colors  
emoticon-happy  
emoticon-sad  
emoticon-smile  
find  
fire
```

```

gift
internet
light-bulb
new
on-off
reminder
save
user-profile

```

La liste est fournie au design-time par `GetModernNotificationDefaultIconNames`.

4.7 Unite design-time

Une unite separee `notify_Design.pas` a ete creee pour isoler le code necessitant `designide`, `DesignIntf`, `DesignEditors` et `ToolsAPI`.

Cette unite effectue l'enregistrement du composant :

```
RegisterComponents('Modern Notification', [TModernNotification]);
```

Elle ajoute aussi les editeurs de proprietes :

- `IconFileName` : bouton `...` ouvrant une boite `TOpenDialog`.
- `DefaultIconName` : liste deroulante des icones incorporees.
- `Version` : propriete en lecture seule.

Le package a donc ete mis a jour avec la dependance `designide` et l'unite `notify_Design.pas`.

4.8 Icone du composant et splash RAD Studio

Une ressource design-time `modern_notification_design.rc` a ete ajoutee pour l'icone du composant :

```

TMODERNNOTIFICATION BITMAP "images\\flag-green.bmp"
TMODERNNOTIFICATION_32 BITMAP "images\\flag-green_32.bmp"

```

`TMODERNNOTIFICATION` est le nom attendu par RAD Studio pour associer un bitmap a la classe `TModernNotification`. La ressource `TMODERNNOTIFICATION_32` est utilisee pour le splash/plugin bitmap.

L'inclusion de la ressource design-time est faite dans `notify_Design.pas` :

```
{ $R 'modern_notification_design.res' }
```

4.9 Nettoyage du code

Un passage de nettoyage a ete effectue apres les ajouts principaux. Les fonctions encore presentes sont utilisees directement ou indirectement par le composant, les proprietes publiees ou l'initialisation design-time.

Les unites inutiles detectees ont ete supprimees :

- `Winapi.WinRT` retire de `notify_Unit.pas`.
- `System.TypeInfo` retire de `notify_Design.pas`.

Apres nettoyage, KAI a relance une compilation complete du package.

4.10 Validations effectuees

Les validations ont ete repetees a chaque etape importante avec `compileProjects`. L'etat final releve pour ce rapport est :

Projet	<code>modern_notification.dproj</code>
Framework	<code>VCL</code>
Plateforme	<code>Win32</code>
Configuration	<code>Release</code>
Compilation	OK, sans message

4.11 Utilisation finale du composant

Utilisation avec icone incorporee

```
ModernNotification1.Title := 'Information';  
ModernNotification1.Body := 'Traitement termine';  
ModernNotification1.DefaultIcon := True;  
ModernNotification1.DefaultIconName := 'button-info';  
ModernNotification1.Show;
```

Utilisation avec icone externe

```
ModernNotification1.Title := 'Alerte';  
ModernNotification1.Body := 'Operation terminee avec erreur';  
ModernNotification1.IconFileName := 'C:\Images\error_64.png';  
ModernNotification1.Show;
```

Utilisation rapide sans composant depose

```
ShowNotification('Information', 'Traitement termine');
```

4.12 Fichiers principaux crees ou modifies

Fichier	Role
<code>notify_Unit.pas</code>	Code runtime du composant, affichage toast, extraction des icones incorporees, proprietes publiees.
<code>notify_Design.pas</code>	Enregistrement design-time, editeurs de proprietes, icone/splash design-time.
<code>modern_notification_resources.rc</code>	Declarations des PNG 64x64 comme ressources <code>RCDATA</code> .
<code>modern_notification_design.rc</code>	Bitmap d'icone du composant et bitmap splash/plugin.
<code>modern_notification.dpk</code>	Package Delphi contenant les unites runtime/design-time et la dependance <code>designide</code> .
<code>modern_notification.dproj</code>	Projet RAD Studio avec les references aux unites et ressources.

4.13 Points importants pour la maintenance

- Apres modification d'une ressource design-time, redemarrer RAD Studio ou reinstaller le package pour voir la nouvelle icone de palette.
- Les toasts Windows utilisent une URI de fichier : les ressources PNG sont donc extraites temporairement avant affichage.
- Pour ajouter une nouvelle icone incorporee, ajouter le PNG 64x64 dans `images` , declarer la ressource dans le `.rc` , puis ajouter l'entree correspondante dans `EmbeddedIcons` .
- Continuer a utiliser KAI pour les fichiers ouverts dans l'IDE afin d'eviter d'ecraser des changements non sauvegardes.